

Esito della risposta PayPrint

11 settembre 2026. La risposta originale sta in risposta-payprint-2026-09-11.md, le domande in mail-payprint-domande-protocollo.md. Ogni affermazione qui sotto è stata verificata sul codice delle due librerie, sul manuale 2.33 e sui log del simulatore; dove una cosa è solo dedotta, è scritto.

In breve

Il blocco è caduto. Durante un incasso la macchina accetta CM — lo dice il fornitore in chiaro: «*Il comando IN accetta solo AN [...] e CM*». L'adattatore di Giano si può disegnare.

Ma la risposta ha fatto emergere tre difetti veri nelle nostre librerie, uno dei quali tocca direttamente i soldi, e ha reso concreto un rischio che fino a ieri era un'ipotesi. Nessuno dei tre si vedeva nel collaudo: il collaudo passa lo stesso.

	Difetto	Gravità
D1	CommitAsync restituisce l'accettazione invece dell'esito, e riporta trattenuto 0,00 su 470 € trattenuti	alta — contabile
D2	Il timeout di 5 minuti abbandona un incasso ancora aperto senza mandare AN: la macchina continua a incassare	alta — denaro
D3	Qualsiasi testo o ER chiude l'attesa dell'incasso: un BUSY provocato da noi perde la transazione	alta — denaro

Aggiornamento dell'11 settembre, sera. D1 e D3 sono corretti nelle due librerie, con i punti 1-8 del capitolo 3: 121 test offline per parte (35 nuovi, di sequenza, contro un finto pagAmico su 127.0.0.1) e 64 passi di collaudo sul simulatore, in C# e in Kotlin. Il passo [IN]+[CM] ora manda CM a incasso aperto e legge il trattenuto dall'esito: 470,00 dove prima leggeva 0,00. **D2 resta aperto**, per scelta: il timeout si toglie dopo le prove su una macchina vera e dopo il keepalive regolato (punto 12). Il dettaglio, e tre cose emerse facendo il lavoro, sono in fondo al capitolo 3.

1. Domanda per domanda

#	Che cosa ha detto	Stato	Che cosa resta
1.1 pausa	nessun delay, « <i>basta che terminino con CR o CR+LF, il simulatore probabilmente ha qualche difficoltà</i> »	parziale	vale per la macchina col terminatore . Sul simulatore va provato
1.2 terminatore	CR o CR+LF	chiusa per la macchina	il manuale 2.33 non lo nomina mai e tutti i suoi esempi ne sono privi (pp. 12, 15, 43, 66): è una regola non scritta
2.1 CM/ST in IN	IN accetta solo AN e CM; ST « <i>verrebbe ignorato</i> », va mandato solo a transazione chiusa	chiusa	lo stesso socket è implicito, non detto. Non dice quanti frame seguono CM
2.2 campi di AN	spiega che amountUnpaid è il resto non erogato per mancanza di monete	aperta	non dice se su AN arrivano collectedAmount e amountUnpaid. La sua frase riguarda il resto di un IN (manuale p. 16), non il rimborso. Nei log non c'è un solo AN con denaro dentro
2.3 spontanei	CMD ERROR = comando sconosciuto; BUSY = macchina impegnata	parziale	sono risposte a comandi, non messaggi spontanei. Su barcode, BT1, EX: silenzio
2.4 incasso aperto	nessun timeout dopo IN; riconnessione accettata dallo stesso IP ; chiusura forzata dal pannello (RESTO + password) → arriva AN; non mettere timeout, I2 sconsigliato	parziale	non dice se dopo la riconnessione i frame arrivano sul nuovo socket, né come chiedere lo stato. E L0, il nostro unico strumento di recupero, durante IN è escluso
2.5 amountPaid	non risponde	chiusa dal manuale	il manuale lo definisce già: « <i>Importo erogato totale</i> » (pp. 9, 59). Nel simulatore vale l'erogato di PA ed è 0 sul resto di un IN. Da confermare se comprende il resto
2.6 tetto	9.999,99 € per i contanti, « <i>maggior per il pos</i> »	chiusa per i contanti	per il POS contraddice il manuale : P0 ha 6 cifre fisse (p. 17)
2.7 parziali	a ogni aggiunta di contante, cumulativi	chiusa	un parziale perso non costa nulla
2.8 password	l'erogazione non dovrebbe essere abilitata di default; la password c'è ma non la ricorda	parziale	serve anche per la chiusura forzata dal pannello

Due correzioni a noi stessi. La 2.5 l'avevamo posta come se amountPaid non fosse documentato (quadro-05 lo diceva in chiaro): il manuale lo definisce a p. 59. E il manuale descrive la chiusura forzata con un gesto diverso da quello del fornitore — «*il rettangolo in alto a sinistra*» (p. 19) invece della parola RESTO; probabilmente il firmware è cambiato, ma va chiesto.

2. I difetti, con le prove

D1 — CommitAsync chiude sul frame sbagliato

Corretto l'11 settembre (punto 1 del capitolo 3). Qui sotto il codice com'era.

Dopo un CM arrivano **due** frame CM. Il predicato della libreria chiudeva sul primo:

```
// PagAmicoClient.cs:517, prima della correzione
return seen >= 2 || f.Json?.CommittedAmount is not null;
```

Ma il primo frame, l'accettazione, porta già il campo — valorizzato a zero, non assente. Dal log del simulatore (collaudo-2026-09-02.log):

Riga	Frame	errorCode	collectedAmount	committedAmount
667	accettazione	OK	470	0.0
668	la libreria scrive: «trattenuto=0,00»			
670	esito finale	(vuoto)	470	470.0

Il frame finale arriva quando nessuno lo attende più. Stesso difetto in Kotlin (PagAmicoClient.kt:412). Il passo di collaudo [IN]+[CM] è sempre risultato OK: **non ha mai dimostrato un commit**.

Sul campo giusto c'è una contraddizione nelle fonti: il fornitore e il diagramma di p. 13 dicono collectedAmount, il testo di p. 44 dice committedAmount. Nel frame finale il simulatore li valorizza entrambi, quindi per l'importo va bene uno o l'altro. **Quello che distingue i due frame è errorCode: OK nell'accettazione, vuoto nel finale** — e questo il manuale non lo documenta.

D2 — il timeout abbandona l'incasso

Ancora aperto, per scelta: va fatto dopo le prove (punto 12 del capitolo 3).

TransactionTimeout vale 5 minuti (PagAmicoClient.cs:74, .kt:97). Allo scadere la libreria smette di attendere e basta: l'AN parte solo se si annulla il token, non per timeout. La macchina resta in IN, il cliente può continuare a inserire denaro. Dall'11 settembre i frame successivi non li prende più il primo comando che passa: arrivano all'evento dei frame orfani (punto 6), dove l'adattatore li può salvare. Ma il gestionale registrerebbe comunque la vendita come fallita mentre la macchina incassa.

Il fornitore dice di non mettere timeout. **Toglierlo però non è una riga**: durante IN non si può mandare ST, quindi l'unico segnale di vita è il keepalive TCP — che oggi è acceso con i valori di sistema (PagAmicoClient.cs:151, .kt:162), cioè **prima sonda dopo due ore**. Un cavo staccato a metà incasso non si vede per ore. Prima serve il keepalive regolato. Lo sblocco delle attese Kotlin alla caduta della connessione, che era l'altro prerequisito, è fatto (punto 7).

Aggiornamento del 14 settembre: keepalive regolato. Nuove impostazioni KeepAliveTime, KeepAliveInterval, KeepAliveRetryCount (C#) e keepAliveTimeSec, keepAliveIntervalSec, keepAliveRetryCount (Kotlin), default **10 s, 2 s, 5 sonde**: una caduta si vede in circa 20 s. La diagnostica dice alla connessione cosa è stato applicato. Limiti: in C# sui target netstandard2.0/net47 il numero di sonde resta quello di Windows (10); in Kotlin la regolazione richiede una JVM che la supporti — su Windows **17.0.14 e 17.0.20 sì, 17.0.8 no** (verificati) — e su Android non è disponibile: resta il keepalive di sistema e la diagnostica lo segnala. Test offline in entrambe le librerie (oggi 170 in C#, 171 in Kotlin). **Non ancora provato con un cavo staccato**: è la prova 10 di checklist-macchina-reale.md. Il timeout di 5 minuti resta finché quella prova non conferma.

D3 — un testo qualsiasi chiude l'incasso

Corretto l'11 settembre (punti 2-4 del capitolo 3). Qui sotto il codice com'era.

```
// PagAmicoClient.cs:389-398, identico in Kotlin a :301-311, prima della correzione
if (f.IsText) return true;          // qualsiasi testo
... r.Equals("ER", ...)            // qualsiasi ER
```

La risposta del fornitore trasforma il rischio in fatto: BUSY è ciò che la macchina risponde a un comando mandato mentre incassa. Il lock dei comandi blocca solo quelli che attendono risposta; **sedici metodi pubblici per libreria** inviano senza lock anche a incasso aperto — display, immagini, stampa senza conferma, RI, SendRawAsync. Ognuno provocherebbe un BUSY, che chiuderebbe l'attesa con un'eccezione mentre la macchina continua a incassare.

Il predicato giusto lavora in **due fasi**. Prima dell'ok di accettazione, un testo vuol dire IN rifiutato (BUSY, CMD ERROR) e chiude. Dopo l'ok, chiudono solo IN, AN e il CM finale; un testo è la risposta a qualcos'altro, va registrato e non tocca l'incasso.

In più: tre situazioni diverse — BUSY in risposta all'IN (non ha incassato nulla), BUSY in risposta a un invio laterale (sta ancora incassando), caduta di rete (forse sta incassando) — escono tutte con la stessa PagAmicoException generica. L'adattatore non può distinguere «non ha incassato» da «forse sta incassando».

3. Che cosa cambia nelle librerie

In ordine di lavoro. Tutto va fatto **nelle due librerie**, con gli stessi nomi.

Senza macchina, subito:

1. **D1** — il CM finale è quello con `errorCode` diverso da OK (o il secondo). Importo letto da `collectedAmount`, con `committedAmount` come controllo.
2. **D3** — predicato di transazione in due fasi; riconoscimento tipizzato di BUSY (testo, o ER con `E100/errorType 99`, p. 61); eccezioni distinte per «rifiutato», «occupato», «connessione persa».
3. CM **durante l'incasso** — via laterale fuori dal lock, come AN. Un solo comando di chiusura per incasso: vince il primo, un CM rifiutato (NO) libera il posto. Se l'IN non è ancora accettato, la chiusura aspetta l'OK.
4. **Invii bloccati a incasso aperto** — tutto ciò che non è AN o CM lancia subito un'eccezione, invece di accodarsi senza limite o di provocare un BUSY. Stato `IsCollecting` esposto.
5. **Parziali** — solo i frame p arrivano al chiamante (oggi passa anche l'OK); in C# niente `Progress<T>` interno, che non garantisce l'ordine.
6. **Frame orfani** — un evento per i frame che arrivano quando nessuno li attende: dopo un timeout o una caduta, o l'AN di una chiusura forzata. Portano importi: l'adattatore li deve ascoltare e salvare.
7. **Kotlin** — alla caduta della connessione le attese falliscono, come in C#.
8. **Test offline di sequenza** per il nuovo predicato: OK p BUSY p IN; OK CMD ERROR IN (lo screenshot di p. 12 mostra tre CMD ERROR dopo un OK); BUSY prima dell'OK; CM/OK seguito da CM finale; CM/NO.

Fatto l'11 settembre: punti 1-8, nelle due librerie, con gli stessi nomi.

#	Come è stato fatto
1	Il CM finale è quello con <code>errorCode</code> diverso da OK e da NO; se ne arrivassero due con OK, chiude il secondo. Il trattenuto si legge in <code>collectedAmount</code> ; se <code>committedAmount</code> è diverso, la diagnostica lo segnala
2	Predicato in due fasi (<code>ClassifyCollection / classifyCollection</code>). <code>PagAmicoFrame.IsBusy</code> riconosce il testo BUSY e l'ER con <code>E100/errorType 99</code> . Quattro eccezioni nuove: <code>PagAmicoRejectedException</code> (rifiutato, non ha incassato), <code>PagAmicoBusyException</code> (sottotipo: rifiutato perché impegnata), <code>PagAmicoConnectionLostException</code> con <code>MaybeCollecting</code> , <code>PagAmicoCollectionOpenException</code> (punto 4)
3	<code>CommitAsync / commit()</code> e <code>CancelAsync / cancelOperation()</code> a incasso aperto passano per la via laterale; la prima chiusura prenota il posto, la seconda lancia; CM/NO lo libera, e un annullo chiesto nel frattempo col token parte allora. Prima dell'OK la chiusura aspetta l'accettazione. <code>CancelAsync</code> a incasso aperto restituisce l'esito dell'incasso
4	<code>IsCollecting / isCollecting</code> ; ogni altro invio lancia <code>PagAmicoCollectionOpenException</code> prima di trasmettere, ricontrollato sotto il lock di scrittura
5	Al chiamante arrivano solo i p, chiamati in modo sincrono dal thread di ricezione
6	<code>OrphanFrame / onOrphanFrame</code> : ogni frame che nessuna attesa riconosce
7	Kotlin: le attese stanno in un registro, come in C#, e alla caduta falliscono con <code>PagAmicoConnectionLostException</code> . In più l'incasso vive nello scope del client: se il chiamante viene cancellato l'attesa continua e l'esito arriva a <code>onOrphanFrame</code> (prima andava perso)
8	35 asserzioni per parte, identiche, in <code>SequenceTests.cs</code> e <code>SequenceTest.kt</code> ; il client vero parla con un finto <code>pagAmico</code> su 127.0.0.1 dentro il processo di test

Aggiornati anche i due banchi (il bottone [AN] usa solo `CancelAsync / cancelOperation()`, i frame orfani finiscono nel log, il trattenuto si legge da `collectedAmount`) e il passo [IN]+[CM] dei due collaudi, che ora usa `CollectCashAsync / collectCash` e manda CM a incasso aperto.

Tre cose emerse facendo il lavoro, da tenere presenti:

- **Un CM in ritardo sul simulatore.** Nel collaudo C# un terzo frame CM, tutto a zero, è arrivato parecchio dopo il commit, all'inizio del passo [P2]: l'ha preso come risposta l'attesa di un ST, e la vera risposta a ST è finita fra gli orfani. Il passo è andato bene per caso. Nel collaudo Kotlin non si è visto. Due conseguenze: quanti frame seguono CM va chiesto anche per questo (domanda 1 della telefonata); e i comandi **semplici** chiudevano sul primo frame qualunque (AnyFrame). **Corretto nella seconda passata**, qui sotto.
- **Un ER dopo l'OK non chiude più l'incasso.** È la regola del punto 2, ma se la macchina chiudesse davvero un incasso con un ER (guasto, banconota incastrata: domanda 5 della telefonata) l'attesa resterebbe aperta fino al timeout — e quando D2 lo toglierà, per sempre. L'ER intanto arriva agli orfani. La risposta alla domanda 5 decide se l'ER va aggiunto ai frame che chiudono.
- **Se l'incasso si chiude prima dell'esito di un CM** (arriva IN o AN), CommitAsync fallisce con un'eccezione che lo dice, e l'eventuale risposta al CM arriva agli orfani. L'esito vero dell'incasso è quello restituito da CollectCashAsync.

Seconda passata, stessa sera: i punti che i test non coprivano. Nelle due librerie, con gli stessi nomi; i test offline passano da 121 a **168 per parte**, i collaudi restano 64 su 64.

- **Comandi semplici:** attendono la propria risposta — il proprio codice (ST su ST), un OK, o un errore — come fa il simulatore con ogni comando del collaudo; il resto va agli orfani. Lo resta libero, perché rimanda l'ultimo JSON. Nel collaudo successivo il CM in ritardo si è ripresentato, proprio mentre ST aspettava: è finito fra gli orfani e ST ha ricevuto la sua risposta.
- **Registro su file, un difetto corretto:** in C# il nome del mutex fra processi si calcolava una volta sola, sul file del primo giorno. Dopo mezzanotte un processo avviato ieri e uno avviato oggi scrivevano lo stesso file con due mutex diversi, e le righe potevano sovrasciversi. Ora il nome segue il file del giorno. Il test che lo verifica **fallisce togliendo la correzione**.
- **Registro su file in Kotlin:** il test con due scrittori sullo stesso file passa anche senza mutex, perché l'append di Java è quello del sistema operativo. La divergenza con C# pesa quindi meno di come era descritta; fra due processi veri non è provato.
- **Test nuovi sull'incasso:** annullo prima dell'OK; PO, IM, I2; chiusura forzata dal pannello (AN con halted TRUE, senza che il client mandi nulla); caduta mentre il CM aspetta l'OK (il CM non parte, l'incasso non risulta accettato); Disconnect() a incasso accettato; timeout dell'incasso (lo stato si libera, i frame successivi vanno agli orfani: la macchina invece resta aperta, ed è D2); caduta dopo un annullo.
- **Test nuovi sull'invio:** la pausa di 80 ms con la configurazione di default, il terminatore, i due incapsulamenti delle immagini byte per byte. E il registro su file (cambio di giorno, ripulitura, troncamento), il comando DI, i codici di errore, il riconoscimento di BUSY.

Col simulatore:

1. **Terminatore** — default a CR, **dopo** averlo provato. I banchi WinForms e Compose passano sempre il valore della loro casella, oggi vuota: cambiando solo il default della libreria continuerebbero a provare senza terminatore. Vanno aggiornati anche loro.
2. **Prova «raffica»** — due comandi in una sola scrittura, con e senza CR. Oggi senza terminatore il secondo si perde.

Aggiornamento del 14 settembre. Punti 9 e 10 fatti. Default del terminatore a CR nelle due librerie e nei due banchi; collaudo 64/64 con CR sia a 80 ms sia a 0 ms, in C# e in Kotlin. La raffica non perde più comandi sul simulatore attuale, **nemmeno senza terminatore**: il difetto non si riproduce. Dettagli in `prova-terminatore-cr-2026-09-14.md`.

Con una macchina vera (quella di Viglione da remoto, o la nostra):

1. **Pausa** — resta configurabile; il default passa da 80 ms a 0 solo quando la raffica con CR regge sulla macchina.
2. **D2** — timeout dell'incasso infinito di default, separato da quello di erogazioni e commit; **dopo** il keepalive regolato.
3. **Riconnessione** — un'API che riprenda l'attesa di un incasso aperto. Aspetta le risposte della telefonata.

4. Che cosa cambia per Giano

- **L'adattatore si può disegnare.** Era il vincolo che bloccava tutto.
- **L'inversione contabile è confermata, non risolta.** CM = incasso **accettato** come pagamento. Giano oggi, con `CancelPayment(false)`, trattiene e annulla. La decisione resta da prendere.
- **Niente timeout nella finestra di pagamento.** Giano già non ne ha: il consiglio del fornitore è coerente con quello che c'è.
- **Heartbeat solo fra un incasso e l'altro.** Durante l'incasso, keepalive TCP.
- **La chiusura forzata dal pannello** arriva come AN con `halted` a TRUE (p. 19): Giano deve saperla distinguere da un annulllo chiesto da lui.
- **Il display della cassa durante l'incasso è escluso** — e Giano oggi non lo usa, quindi non costa niente.
- **Un CM può trattenere più del dovuto.** Sul simulatore, 80 € richiesti e 470 € trattenuti senza resto (log 2/9, riga 667): il client non aveva ricevuto nessun parziale prima del CM, e non poteva sapere che l'importo era già superato. Per rendere l'eccedenza serve PA, che il fornitore dice disabilitato di default.

5. Da chiedere in telefonata

In ordine di peso.

1. **Dopo un CM durante IN:** quanti frame, uno o due? L'importo va letto in collectedAmount o committedAmount? Come si distingue con certezza l'accettazione dall'esito?
2. **CM a importo già superato:** la macchina rende il resto o trattiene tutto?
3. **AN con denaro dentro:** arrivano collectedAmount e changeReturn? Se non riesce a rendere tutto, quale campo lo dice?
4. **BUSY:** in che forma arriva — il testo BUSY, un ER con E100, altro? E un ST mandato durante l'incasso riceve BUSY o niente?
5. **Dopo l'OK di un IN** può arrivare un ER che chiude l'incasso (guasto, banconota incastrata)? Il denaro inserito viene reso?
6. **Riconnessione dallo stesso IP:** i parziali e l'esito arrivano sul nuovo socket? Sul nuovo socket si possono mandare AN e CM? Una connessione da un altro IP viene rifiutata sempre, o solo durante un incasso? (conta per il DHCP della cassa)
7. **Chiusura forzata:** il denaro viene reso al cliente? Il gesto è la parola RESTO o il rettangolo in alto a sinistra del manuale? Qual è la password, ed è la stessa dell'erogazione?
8. **Durante un incasso lungo,** senza ST e senza timeout, come ci accorgiamo che la macchina non è più raggiungibile? Il firmware regge un keepalive TCP a intervalli brevi?
9. **Terminatore:** dopo i pacchetti immagine SF/SI serve un CR? E PTPRDT, che può contenere CR e LF?
10. amountPaid comprende il resto dato durante un IN, o solo quanto erogato con PA?
11. (*minore*) **POS oltre 9.999,99 €:** con quale formato, visto che PO ha 6 cifre fisse?

E accettare la sua offerta: **il collegamento alla sua macchina**, per le prove dei punti 1-6.